

EasyVolcap: Accelerating Neural Volumetric Video Research

Zhen Xu
Tao Xie
Sida Peng
Haotong Lin
zhenx@zju.edu.cn
xbillowy@gmail.com
pengsida@zju.edu.cn
haotongl@outlook.com
Zhejiang University
China

Qing Shuai
Zhiyuan Yu
Guangzhao He
s_q@zju.edu.cn
zyuaq@ust.hk
alexhe2000@zju.edu.cn
HKUST
China
Zhejiang University
China

Jiaming Sun
Hujun Bao
Xiaowei Zhou*
suenjiaming@gmail.com
bao@cad.zju.edu.cn
xwzhou@zju.edu.cn
Image Derivative Inc.
China
Zhejiang University
China

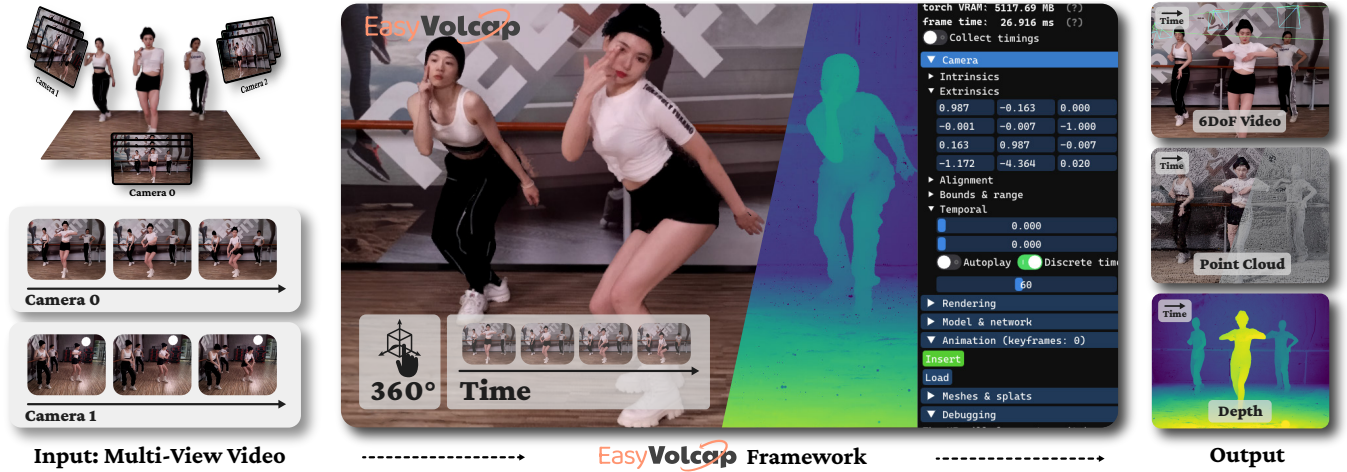


Figure 1: EasyVolcap is a Python & PyTorch library for accelerating volumetric video research, particularly in the area of neural dynamic scene representation, reconstruction, and rendering. Given multi-view video input, EasyVolcap streamlines the process of data preprocessing, 4D reconstruction, and rendering of dynamic scenes.

CCS CONCEPTS

• Computing methodologies → Image-based rendering.

KEYWORDS

Novel view synthesis

ACM Reference Format:

Zhen Xu, Tao Xie, Sida Peng, Haotong Lin, Qing Shuai, Zhiyuan Yu, Guangzhao He, Jiaming Sun, Hujun Bao, and Xiaowei Zhou. 2023. EasyVolcap: Accelerating Neural Volumetric Video Research. In *SIGGRAPH Asia 2023 Technical Communications (SA Technical Communications '23)*, December 12–15, 2023, Sydney, NSW, Australia. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3610543.3626173>

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SA Technical Communications '23, December 12–15, 2023, Sydney, NSW, Australia
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0314-0/23/12...\$15.00
<https://doi.org/10.1145/3610543.3626173>

1 INTRODUCTION

Volumetric video is a technology that digitally records dynamic events such as artistic performances, sporting events, and remote conversations. When acquired, such volumetry can be viewed from any viewpoint and timestamp on flat screens, 3D displays, or VR headsets, enabling immersive viewing experiences and more flexible content creation in a variety of applications such as sports broadcasting, video conferencing, gaming, and movie productions. With the recent advances and fast-growing interest in neural scene representations for volumetric video, there is an urgent need for a unified open-source library to streamline the process of volumetric video capturing, reconstruction, and rendering for both researchers and non-professional users to develop various algorithms and applications of this emerging technology.

In this paper, we present EasyVolcap, a Python & PyTorch library for accelerating neural volumetric video research with the goal of unifying the process of multi-view data processing, 4D scene reconstruction, and efficient dynamic volumetric video rendering. Given the insight into the most popular paradigm of dynamic scene reconstruction and rendering methods, we build a well-structured unified pipeline for 4D scene reconstruction, which is composed of a 4D-aware feature embedder and an MLP-based regressor as shown

in Figure 2. Given the spacetime coordinates, the feature embedder maps them to a high-dimensional feature vector, which is then passed through the geometry and appearance MLPs to regress the final output density and color. Moreover, we provide a set of easy-to-use tools for 4D volumetric video researchers like a native high-performance viewer marrying the extensibility of Python and the power of OpenGL and CUDA, and a robust training and evaluation loop for multi-view datasets. Compared to NeRFStudio [Tancik et al. 2023], which focuses on NeRF-based [Mildenhall et al. 2021] static scene modeling, EasyVolcap’s data-loading procedure, unified pipeline, and other logistic systems are all specifically designed for dynamic 3D scenes as shown in Table 1. Note that EasyVolcap trivially supports taking a static 3D dataset or a monocular dynamic dataset as input since they can be considered as special cases of the volumetric video with only one frame or only one camera. We hope that by building a readily accessible open-source library on 4D volumetric video, future researchers on this topic could more easily express their creativity, develop brand-new algorithms, and discover groundbreaking insights.

2 FRAMEWORK DESIGN

We develop a unified framework after studying the recurring patterns of prominent 4D volumetric video methods where components can even be directly swapped from the command line. This unified pipeline includes a 2D and 3D sampler, a set of space-time 4D embedding structures, a deformation module to apply flow-like tracking on the underlying 3D structure, an appearance or transient embedding where appearance-only tunes could be applied, and finally, a set of regressors to convert embedded features to the final output. Figure 2 provides a detailed illustration of the framework’s unified pipeline. Moreover, when a novel algorithm requires a completely different network organization, it is also trivially easy to swap out the core components of the fixed pipeline.

Input & dataset representation. EasyVolcap provides a centralized but inheritable dataset management system where the most basic

Table 1: Comparison of different neural volumetric video frameworks. NeRFStudio [Tancik et al. 2023] and SDFStudio [Yu et al. 2022] are designed for static scenes and do not support multi-view data input. NerfAcc [Li et al. 2023] and Kaolin-Wisp [Takikawa et al. 2022] do not support playback of dynamic volumetric content. Our framework supports both multi-view video datasets and playback of dynamic volumetric videos.

Framework	Multi-View Video Dataset	Volumetric Video Player
NerfStudio [Tancik et al. 2023]	✗	✓
SDFStudio [Yu et al. 2022]	✗	✗
NerfAcc [Li et al. 2023]	✗	✗
Kaolin-Wisp [Takikawa et al. 2022]	✗	✗
Ours	✓	✓

form input is an unstructured image tensor of $[n_{frame}, n_{view}]$. Numerous optimizations like dataset sharding for multi-gpu training, input data compression (which is never a problem until you add a *time* dimension) with a custom addressable but unstructured tensor class, flexible input formats where pixel mask, pixel importance, and visual-hull-based bounding box determination are all a single switch away. One can also easily optimize the camera parameters thanks to the optimizable camera residual applied before sampling.

Point & ray sampling strategy. Typical NeRF-based [Mildenhall et al. 2021] rendering requires the conversion from camera rays to points sampled on the ray before querying the network and performing volume rendering. The sampler family of EasyVolcap unifies this process with a corpus of point samplers like a uniform sampler, a disparity-based sampler [Barron et al. 2022], a coarse-to-fine importance sampler [Mildenhall et al. 2021], a human shape guided [Peng et al. 2022] sampler and a cost-volume-based depth-guided sampler [Lin et al. 2022]. Optimizable parameters like the 2D feature extractors of cost-volume builder are also easily introduced in this stage.

Space-time feature embedding. At the core of the main-stream volumetric representations is a set of 4D-aware encodings [Fridovich-Keil et al. 2023], either computed from representative implicit structures or fast explicit proxies. EasyVolcap embraces this concept by providing a KPlanes-style [Fridovich-Keil et al. 2023] decomposed feature embedder and a composable 4D embedder (positional encoding [Mildenhall et al. 2021], multi-resolution hash encoding [Müller et al. 2022] and latent-codes [Peng et al. 2022]). The process of implementing and benchmarking a novel 4D representation is also streamlined thanks to EasyVolcap’s robust registration and configuration system.

Deformation & flow composition. Several other prominent volumetric video research proposed components to model the deformation of the dynamic scene using scene flow [Wang et al. 2023], deformation fields [Park et al. 2021a] or hyper-networks [Park et al. 2021b]. As an effective way to model the dynamic nature of the volumetric video, EasyVolcap also provides a set of deformation modules that can be easily applied along with their respective regularizations.

Appearance & transient feature embedding. [Zhang et al. 2020] propose to model the dynamic appearance of a time-varying scene with an appearance-specific latent embedding or tailored transient functions. Thanks to the modular nature of EasyVolcap’s unified pipeline, such transient embedding is easily applied at the appearance embedding stage.

Output Regressor. Given the embedded feature of a four-dimensional coordinate, EasyVolcap provides a set of physical property regressors to differentially translate a neural descriptor in the 4D space to an actual output, such as volume density [Mildenhall et al. 2021], signed distance [Wang et al. 2021a], RGB color [Mildenhall et al. 2021], or Spherical Harmonics coefficients [Yu et al. 2021].

3 HIGHLIGHTED ALGORITHMS

Utilizing the unified and extensible research framework provided by EasyVolcap, we incorporate state-of-the-art volumetric rendering

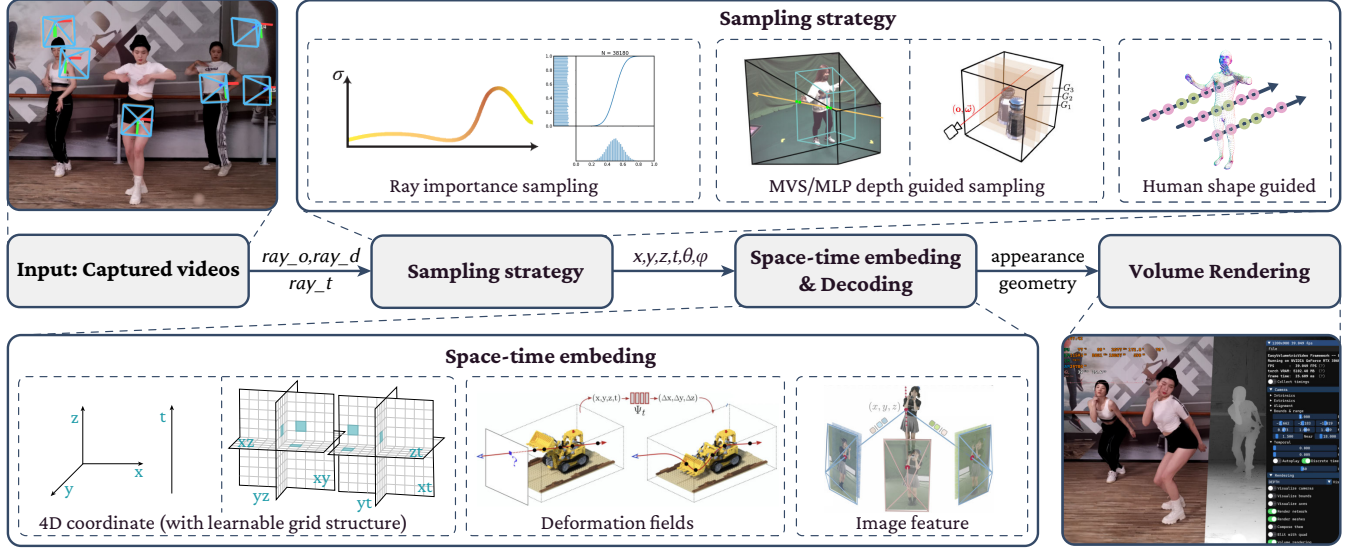


Figure 2: Framework design of EasyVolcap. Given captured videos, EasyVolcap samples space-time rays for rendering. Ray sampling strategies are applied to generate 4D point coordinates. These points are then encoded with space-time embedders, decoded for physical values (density, color, etc.), and then fed into the volume renderer to obtain the final rendering.

algorithms and extract their reusable components as building blocks and inspirations for future research. Here we briefly introduce two of the most prominent ones.

Efficient neural radiance fields. ENeRF [Lin et al. 2022] is a generalizable neural radiance field method. Combining the power of cost-volume-based depth estimation and generalizability of image features (Image-Based Rendering), ENeRF achieves unprecedented performance and generalizability on both static and dynamic scenes. We extract the reusable MVSNerF-style [Chen et al. 2021] cost-volume-based depth estimation module, IBNet-style [Wang et al. 2021b] appearance learning and a depth-guided NeRF sampling pipeline like DONeRF [Neff et al. 2021].

Realtime rendering of dynamic volumetric video. 4K4D [Xu et al. 2023] is a real-time 4D view synthesis method developed using the EasyVolcap framework. We achieve 60FPS at 4K resolution on rendering of neural volumetric video by introducing hardware-accelerated differentiable depth-peeling algorithm on point clouds [Kerbl et al. 2023; Zhang et al. 2022]. Through the parameter-sharing of the 4D structures, the geometry and appearance feature effectively fuse 4D information present in the multi-view video. Thanks to the compactness and expressiveness of the hybrid structure, we can pre-compute features on the explicit geometry during inference, achieving unprecedented rendering speed. The teaser and video of the brief are produced by this algorithm.

Other supported methods. EasyVolcap also supports a wide range of other algorithms including [Fridovich-Keil et al. 2023; Mildenhall et al. 2021; Müller et al. 2022; Park et al. 2021a; Tancik et al. 2023; Wang et al. 2021a; Zhang et al. 2022] with much more coming.

4 FRAMEWORK UTILITIES

4.1 High-Performance Native Viewer

EasyVolcap provides a high-performance native viewer that delivers the rendered content of various custom algorithms to the user's screen with low latency, high throughput, and extensibility by incorporating the easy-to-use Python language to define control elements. A comparison with other common open-source implementations can be found in Table 2.

CPU-GPU communication. EasyVolcap's viewer implementation fully harnesses the power of asynchronous CUDA kernel launching to overlap the slow Python-based user interface with GPU-side network rendering. Moreover, by directly utilizing the CUDA-Graphics interface, EasyVolcap copies the rendered content stored in a PyTorch tensor directly to the frame buffer of the OpenGL context to be displayed on the screen. This design and implementation choice avoids a heavy GPU-CPU-GPU copy chain and their respective synchronization points, greatly enhancing the throughput and reducing the latency and overhead of the viewer.

Memory Management. Although a single frame being rendered is usually small enough to be stored directly in the VRAM, most dynamic volumetric video representations are too large to directly fit onto the VRAM of the Graphics Card. Taking inspiration from other video playback software [pot [n. d.]], EasyVolcap heuristically swaps out the VRAM with the immediately following frames to be played on the pinned main memory with asynchronous and non-blocking copy [mem [n. d.]] to satisfy both the computing and PCIe hardware, achieving maximum rendering speed.

4.2 Logistics Systems

Robust configuration system. EasyVolcap extends the configuration system of XRNeRF [XRNeRF 2022] by incorporating their registration system and extending their yaml-based configuration

Table 2: Comparison of different viewers. The web-based viewer adopted by NeRFStudio [Tancik et al. 2023] shows great extensibility, however, the unavoidable network transfer increases latency. Our native viewer excels in all these aspects thanks to the seamless integration of CUDA memory copy and the viewer’s OpenGL context.

Viewer	Method	Low Latency	Async Drawing	Extensibility	Cross-Platform
WebViewer	NeRFStudio[Tancik et al. 2023]	✗	✗	✓	✓
C++-based Viewer	InstantNGP[Müller et al. 2022], 3D Gaussian[Kerbl et al. 2023]	✓	✓	✗	✓
Python-based	ENeRF[Lin et al. 2022], torch-ngp[Tang 2022]	✓	✗	✓	✓
Ours		✓	✓	✓	✓

Table 3: Comparison of the configuration system. yacs is the most common among open-source papers [Peng et al. 2022], however, it lacks an extensible and dynamic file type support, making it hard to scale. XRNeRF’s [XRNeRF 2022] register system shows great potential, but our configuration system handles inheritance between files better than theirs.

Config System	Method	yaml	python	json	Command Line	Inheritance	Multi-Inheritance
yacs	ENeRF[Lin et al. 2022]	✓	✗	✗	✓	✗	✗
dataclasses	NeRFStudio[Tancik et al. 2023]	✗	✓	✗	✓	✓	✗
gin	MipNeRF-360[Barron et al. 2022]	✗	✗	✗	✗	✗	✗
register	XRNeRF[XRNeRF 2022]	✓	✓	✓	✓	✓	✗
Ours		✓	✓	✓	✓	✓	✓

system with the support of the command line tab-complete. Inheritance from multiple different config files is also supported to plug in various different settings conveniently, along with an extended reference system that supports constructing configurations using the dynamic values in other config files. Such adaptation allows for an easy plug-and-play experience to swap out new datasets, entirely new algorithms, or just replace a hyperparameter setting directly from the command line to perform new experiments. A comparison with other popular configuration frameworks is provided in Table 3.

ACKNOWLEDGMENTS

The authors would like to acknowledge support from NSFC (No. 62172364), Information Technology Center, and State Key Lab of CAD&CG, Zhejiang University.

REFERENCES

- [n. d.]. Memory Management. <https://docs.nvidia.com/cuda/cuda-runtime-api/index.html>
- [n. d.]. PotPlayer 230405. <https://daumpotplayer.com/>
- Jonathan T. Barron, Ben Mildenhall, Dor Verbin, Pratul P. Srinivasan, and Peter Hedman. 2022. Mip-NeRF 360: Unbounded Anti-Aliased Neural Radiance Fields. *CVPR* (2022).
- Anpei Chen, Zexiang Xu, Fuqiang Zhao, Xiao Shuai Zhang, Fanbo Xiang, Jingyi Yu, and Hao Su. 2021. Mvsnerf: Fast generalizable radiance field reconstruction from multi-view stereo. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 14124–14133.
- Sara Fridovich-Keil, Giacomo Meanti, Frederik Rahbæk Warburg, Benjamin Recht, and Angjoo Kanazawa. 2023. K-planes: Explicit radiance fields in space, time, and appearance. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 12479–12488.
- Bernhard Kerbl, Georgios Kopanas, Thomas Leimkuehler, and George Drettakis. 2023. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. *ACM Transactions on Graphics (TOG)* 42, 4 (2023), 1–14.
- Ruilong Li, Hang Gao, Matthew Tancik, and Angjoo Kanazawa. 2023. NerfAcc: Efficient Sampling Accelerates NeRFs. *arXiv preprint arXiv:2305.04966* (2023).
- Haotong Lin, Sida Peng, Zhen Xu, Yunzhi Yan, Qing Shuai, Hujun Bao, and Xiaowei Zhou. 2022. Efficient Neural Radiance Fields for Interactive Free-viewpoint Video. In *SIGGRAPH Asia Conference Proceedings*.
- Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. 2021. Nerf: Representing scenes as neural radiance fields for view synthesis. *Commun. ACM* 65, 1 (2021), 99–106.
- Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. 2022. Instant neural graphics primitives with a multiresolution hash encoding. *ACM Transactions on Graphics (ToG)* 41, 4 (2022), 1–15.
- Thomas Neff, Pascal Stadlbauer, Mathias Parger, Andreas Kurz, Joerg H. Mueller, Chakravarty R. Alla Chaitanya, Anton Kaplanyan, and Markus Steinberger. 2021. DOnERF: Towards Real-Time Rendering of Compact Neural Radiance Fields using Depth Oracle Networks. In *Computer Graphics Forum*, Vol. 40. Wiley Online Library, 45–59.
- Keunhong Park, Utkarsh Sinha, Jonathan T. Barron, Sofien Bouaziz, Dan B. Goldman, Steven M. Seitz, and Ricardo Martin-Brualla. 2021a. Nerfies: Deformable neural radiance fields. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 5865–5874.
- Keunhong Park, Utkarsh Sinha, Peter Hedman, Jonathan T. Barron, Sofien Bouaziz, Dan B. Goldman, Ricardo Martin-Brualla, and Steven M. Seitz. 2021b. Hypernerf: A higher-dimensional representation for topologically varying neural radiance fields. *arXiv preprint arXiv:2106.13228* (2021).
- Sida Peng, Zhen Xu, Juntong Dong, Qianqian Wang, Shangzhan Zhang, Qing Shuai, Hujun Bao, and Xiaowei Zhou. 2022. Animatable Implicit Neural Representations for Creating Realistic Avatars from Videos. *arXiv preprint arXiv:2203.08133* (2022).
- Towaki Takikawa, Or Perel, Clement Fuji Tsang, Charles Loop, Joey Litalien, Jonathan Tremblay, Sanja Fidler, and Maria Shugrina. 2022. Kaolin Wisp: A PyTorch Library and Engine for Neural Fields Research. <https://github.com/NVIDIAGameWorks/kaolin-wisp>.
- Matthew Tancik, Ethan Weber, Evonne Ng, Ruilong Li, Brent Yi, Terrance Wang, Alexander Kristoffersen, Jake Austin, Kamyar Sahahi, Abhik Ahuja, et al. 2023. Nerfstudio: A modular framework for neural radiance field development. In *ACM SIGGRAPH 2023 Conference Proceedings*. 1–12.
- Jiaxiang Tang. 2022. Torch-ngp: a PyTorch implementation of instant-ngp. <https://github.com/ashawkey/torch-ngp>.
- Peng Wang, Lingjie Liu, Yuan Liu, Christian Theobalt, Taku Komura, and Wenping Wang. 2021a. NeuS: Learning Neural Implicit Surfaces by Volume Rendering for Multi-view Reconstruction. In *NeurIPS*.
- Qianqian Wang, Yen-Yu Chang, Ruojin Cai, Zhengqi Li, Bharath Hariharan, Aleksander Holynski, and Noah Snavely. 2023. Tracking Everything Everywhere All at Once. *arXiv preprint arXiv:2306.05422* (2023).
- Qianqian Wang, Zhicheng Wang, Kyle Genova, Pratul P. Srinivasan, Howard Zhou, Jonathan T. Barron, Ricardo Martin-Brualla, Noah Snavely, and Thomas Funkhouser. 2021b. Ibrnet: Learning multi-view image-based rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 4690–4699.
- XRNeRF. 2022. OpenXRLab Neural Radiance Field Toolbox and Benchmark. <https://github.com/openxrlab/xrnerf>.
- Zhen Xu, Sida Peng, Haotong Lin, Guangzhao He, Jiaming Sun, Yujun Shen, Hujun Bao, and Xiaowei Zhou. 2023. 4K4D: Real-Time 4D View Synthesis at 4K Resolution. (2023).
- Alex Yu, Ruilong Li, Matthew Tancik, Hao Li, Ren Ng, and Angjoo Kanazawa. 2021. Plenotrees for real-time rendering of neural radiance fields. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 5752–5761.
- Zehao Yu, Anpei Chen, Bozidar Antic, Songyou Peng, Apratim Bhattacharya, Michael Niemeyer, Siyu Tang, Torsten Sattler, and Andreas Geiger. 2022. SDFStudio: A Unified Framework for Surface Reconstruction. <https://github.com/autonomousvision/sdfstudio>
- Kai Zhang, Gernot Riegler, Noah Snavely, and Vladlen Koltun. 2020. Nerf++: Analyzing and improving neural radiance fields. *arXiv preprint arXiv:2010.07492* (2020).
- Qiang Zhang, Seung-Hwan Baek, Szymon Rusinkiewicz, and Felix Heide. 2022. Differentiable point-based radiance fields for efficient view synthesis. In *SIGGRAPH Asia 2022 Conference Papers*. 1–12.